Data integration and Big Data S9: COVID-19 Dataset Analysis

M2 Genomics Informatics and Mathematics for Health and Environment

---

Author:

Christophoros Mitsakopoulos

October 29, 2025

Contents

1 Introduction

This report focuses on comparing the performance of MongoDB, Hadoop MapReduce (MR) and Spark MR on a real-world dataset. Given the modest size of the ECDC COVID-19 dataset (61,900 records), the strategy is to benchmark these platforms against three identical, multi-stage aggregation tasks, to ensure that they are compared fairly:

- Sum total cases/deaths and compute Case Fatality Rate (CFR) per country.
- Sum total cases/deaths and compute CFR per continent.
- Sum global daily totals and compute the global daily CFR.

The above tasks were executed with MacOS v15.6.1 (24G90), on an M1 MacBook Air device while connected to power supply, in order to increase CPU voltage and battery throughput. Given the age of the device (4.5 years), expect that this outdated and degraded hardware should be slower than your own device, if willing to reproduce the findings of this report.

The Hadoop-Python streaming scripts are directly adapted from proven logic / code used in Report 2. For MongoDB, the modern Aggregation Pipeline from Report 1 is re-used, as the native `mapReduce()` command is deprecated in the latest versions (ex. v.2.5.8 is intalled in the testing device). In regard to the Spark section, a combination of lessons learned from Reports 1 and 2 were utilised and direct references to the slides have been made.

2 Understanding the data and preparing for analysis

2.1 Preparing MongoDB and observing the schema

Following the instructions of the lectures, the COVID-19 dataset was obtained in JSON format (i.e. `covid19.json`) from the ECDC at: `https://www.ecdc.europa.eu/en/publications-data/download-todays-data-geographic-distribution-covid-19-cases-worldwide`. Note that due to running MongoDB and Hadoop from a Homebrew environment the methodology differs greatly from a Docker based environment, initiating services therefore looks as such: `brewservicesstartmongodb-community`. Nevertheless, to analyse this COVID-19 data, the first step was to import `covid19.json` into a new MongoDB database. The initial import attempt using default `mongoimport` parameters unexpectedly failed. The file was not a standard JSON array but rather a single JSON object containing a key (i.e. "records") which held the array of all 61,900 documents.

mongoimport's default mode expects newline-delimited JSON (NDJSON), where each line is a separate record. When the file was read, it was interpreted as one massive, single-line record. Even attempting to import the file using the `--jsonArray` flag at the end of the import command, yielded a file size error yet again – as the multiple lines in the document were misperceived as a single record.

```
chrismitsacopoulos@Chriss-MacBook-Air Desktop % wc -l covid19.json
  866603 covid19.json
```

Listing 1: Examining the `covid19.json` file to see its contents and understand the errors encountered while attempting `mongoimport`. Observe the unexpectedly large line count (866603), dividing that number by the 13 data fields within each record (as was identified later), yields a number $\approx$ 61900. Therefore, we can conclude that the real issue is that records are not split per line, each data field is on its own line – causing obvious formatting issues.

The solution was to pre-process the file using the jq utility. By using the filter ".records[]", the single line array was "exploded" splitting each data record within it, into a separate lines using the -c (compact) flag; converting the file into the correct NDJSON format.

```
chrismitsacopoulos@Chriss-MacBook-Air ~ % jq -c '.records[]' /Users/
    chrismitsacopoulos/Desktop/covid19.json > /Users/chrismitsacopoulos/
    Desktop/covid19_nd.json
```

Listing 2: " key as reference. Thereby, converting the file into NDJSON which is accepted by the default MongoDB import parameters.]Using `jq` (installed with `brewinstalljq`) to split JSON entries into separate lines using the "records[]" key as reference. Thereby, converting the file into NDJSON which is accepted by the default MongoDB import parameters.

This new file, `covid19_nd.json`, was then imported successfully by mongoimport; with the number of imported records matching the ones advertised on ECDC.

```
chrismitsacopoulos@Chriss-MacBook-Air ~ % mongoimport --db covid_db --
    collection covid_data --file /Users/chrismitsacopoulos/Desktop/covid19_
    nd.json
2025-10-26T17:44:22.190+0100  connected to: mongodb://localhost/
2025-10-26T17:44:23.108+0100  61900 document(s) imported successfully. 0
    document(s) failed to import.
```

Listing 3: Import the `covid19_nd.json` reformatted file into MongoDB under a new `covid_db` database and `covid_data` collection.

```
test> use covid_db
switched to db covid_db
covid_db> db.covid_data.findOne()
{
  _id: ObjectId('68fe4fe6b45b292b059abba9'),
  dateRep: '10/12/2020',
  day: '10',
  month: '12',
  year: '2020',
  cases: 202,
  deaths: 16,
  countriesAndTerritories: 'Afghanistan',
  geoId: 'AF',
  countryterritoryCode: 'AFG',
  popData2019: 38041757,
  continentExp: 'Asia',
  'Cumulative_number_for_14_days_of_COVID-19_cases_per_100000': '6.96865815
    '
}
```

Listing 4: Entering and switching to the `covid_db` we creted just prior, within MongoDB (i.e. `mongosh` service), then projecting a single record to observe its contents / format.

Table 1: COVID-19 dataset field descriptions, example values for each field are the same as the above code listing.

Field Name	Example	Description
_id	`ObjectId(...)`	Record ID automatically generated by MongoDB.
dateRep	`'10/12/2020'`	The date of the report as a string as DD/MM/YYYY.
day	`'10'`	The day of the month as string.
month	`'12'`	The month as string.
year	`'2020'`	The year as string.
cases	`202`	The number of cases reported for the day (numeric).
deaths	`16`	The number of deaths reported for the day (numeric).
countriesAndTerritories	`'Afghanistan'`	Full name of the country or territory.
geoId	`'AF'`	2-letter (ISO 3166-1 alpha-2) country code.
countryterritoryCode	`'AFG'`	3-letter (ISO 3166-1 alpha-3) country code.
popData2019	`38041757`	Total population of the record's country in 2019 (numeric).
continentExp	`'Asia'`	The continent where the country or territory is located.
Cumulative_number...	`'6.968...'`	The 14-day cumulative number of cases per 100,000 people.

2.2 Quality Assurance

To ensure the integrity of the analysis, a JavaScript script was drafted to automate QA. Specifically, this script checks the entire `covid_data` collection for null values, incorrect data types (i.e. `cases` being a string), or malformed data like negative case numbers or inconsistent date strings.

```
print("Starting Data Quality Report for 'covid_data' collection...\n");

print("\nCheck for any records which have a null field; incomplete records");
const nullDocsCount = db.covid_data.aggregate([
    { $addFields: { __values: { $objectToArray: "$$ROOT" } } },
    { $match: { "__values.v": null } },
    { $count: "count" }
]).toArray();

print("\nCheck for any records which have more than a single null field; super incompelte records");
```

```
if (nullDocsCount.length > 0) {
    print(`WARNING: Found ${nullDocsCount[0].count} documents with at least one 'null' value.`);
    let example = db.covid_data.aggregate([
        { $addFields: { __values: { $objectToArray: "$$ROOT" } } },
        { $match: { "__values.v": null } },
        { $limit: 1 }
    ]).next();
    print("Example bad doc:", JSON.stringify(example, null, 2));
} else {
    print("SUCCESS: No documents with 'null' values found.");
}

print("\n'cases' field");
const wrongCasesTypeCount = db.covid_data.countDocuments({ cases: { $not: { $type: "number" } } });

if (wrongCasesTypeCount > 0) {
    print(`WARNING: Found ${wrongCasesTypeCount} documents where 'cases' is NOT a number.`);
    let example = db.covid_data.findOne({ cases: { $not: { $type: "number" } } });
    print("Example bad doc:", JSON.stringify(example, null, 2));
} else {
    print("SUCCESS: All 'cases' fields are numeric.");
}

print("\n'deaths' field type");
const wrongDeathsTypeCount = db.covid_data.countDocuments({ deaths: { $not: { $type: "number" } } });

if (wrongDeathsTypeCount > 0) {
    print(`WARNING: Found ${wrongDeathsTypeCount} documents where 'deaths' is NOT a number.`);
    let example = db.covid_data.findOne({ deaths: { $not: { $type: "number" } } });
    print("Example bad doc:", JSON.stringify(example, null, 2));
} else {
    print("SUCCESS: All 'deaths' fields are numeric.");
}

print("\nnegative numbers");
const negativeValuesCount = db.covid_data.countDocuments({ $or: [{ cases: { $lt: 0 } }, { deaths: { $lt: 0 } }] });

if (negativeValuesCount > 0) {
    print(`WARNING: Found ${negativeValuesCount} documents with negative cases or deaths.`);
    let example = db.covid_data.findOne({ $or: [{ cases: { $lt: 0 } }, { deaths: { $lt: 0 } }] });
    print("Example bad doc:", JSON.stringify(example, null, 2));
} else {
    print("SUCCESS: No negative cases or deaths found.");
}

print("\ndateRep' format (DD/MM/YYYY)");
const badDateFormatCount = db.covid_data.countDocuments({ dateRep: { $not: { $regex: /^\d{2}\/\d{2}\/\d{4}$/ } } });
```

```
58
59 if (badDateFormatCount > 0) {
60     print(`WARNING: Found ${badDateFormatCount} documents where 'dateRep'
       does not match DD/MM/YYYY format.`);
61     let example = db.covid_data.findOne({ dateRep: { $not: { $regex: /^\d
       {2}\/\d{2}\/\d{4}$/ } } });
62     print("Example bad doc:", JSON.stringify(example, null, 2));
63 } else {
64     print("SUCCESS: All 'dateRep' fields match the expected format.");
65 }
```

Listing 5: This script can specifically: use a regex to find dateRep strings that do not match the expected DD/MM/YYYY format, count records with negative values for cases or deaths, count records where cases or deaths are not numeric (ex. string), find records with null values using aggregation. When running this code locally, an ">" operator was used to capture the print statements into `QA_result.txt` for later reference.

```
1  chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % mongosh --db covid_db
       --file QA_covid_data.js
2  Starting Data Quality Report for 'covid_data' collection...
3  Check for any records which have a null field; incomplete records
4
5  Check for any records which have more than a single null field; super
       incompelte records
6  SUCCESS: No documents with 'null' values found.
7
8  'cases' field
9  SUCCESS: All 'cases' fields are numeric.
10
11 'deaths' field type
12 SUCCESS: All 'deaths' fields are numeric.
13
14 negative numbers
15 SUCCESS: No negative cases or deaths found.
16
17 dateRep' format (DD/MM/YYYY)
18 SUCCESS: All 'dateRep' fields match the expected format.
```

Listing 6: Executing the QA JavaScript script. The output confirms the dataset has: no null values, negative counts, or formatting errors.

3 Data Analysis with MongoDB Aggregate; replacement for deprecated MR commands

Since MR functions in MongoDB are now deprecated (legacy), the next best method of attaining the objectives laid out below is with the aggregate lookup commands.

3.1 Total Cases and Deaths per Country with CFR

```
covid_db> db.covid_data.aggregate([ { $group: { _id: "$
    countriesAndTerritories", totalCases: { $sum: "$cases" }, totalDeaths: {
     $sum: "$deaths" } } }, { $project: { _id: 1, totalCases: 1, totalDeaths
    : 1, CFR: { $cond: [ { $gt: ["$totalCases", 0] }, { $multiply: [ { $
    divide: ["$totalDeaths", "$totalCases"] }, 100 ] }, 0 ] } } }, { $sort:
    { totalCases: -1 } }, { $out: "totals_per_country_agg" } ]);

covid_db> db.totals_per_country_agg.findOne()
{
  _id: 'United_States_of_America',
  totalCases: 16256754,
  totalDeaths: 299177,
  CFR: 1.840324335350095
}
```

Listing 7: The following command uses `\protect\T1\textdollargroup` stage to sum the cases and deaths metrics for each unique `countriesAndTerritories` (i.e. country), the highlight of the CFR section is the use of a conditional statement `\protect\T1\textdollarcond` which prevents the division of deaths by cases – if cases are not greater (`\protect\T1\textdollargt`) than zero. The `\protect\T1\textdollarout` statement saves the results to a new collection named `totals_per_country_agg`. Projecting the new `totals_per_country_agg` collection with the `findOne()` formula.

```
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % mongoexport --db covid
    _db --collection totals_per_country_agg --out totals_per_country.json --
    jsonArray
2025-10-27T00:03:54.947+0100  connected to: mongodb://localhost/
2025-10-27T00:03:54.957+0100  exported 214 records
```

Listing 8: Exporting the new `totals_per_country_agg` collection to file, to visualise and analyse the data at a later stage.

3.2 Total Cases and Deaths per Continent with CFR

```
covid_db> db.covid_data.aggregate([ { $group: { _id: "$continentExp",
    totalCases: { $sum: "$cases" }, totalDeaths: { $sum: "$deaths" } } }, {
    $project: { _id: 1, totalCases: 1, totalDeaths: 1, CFR: { $cond: [ { $gt
    : ["$totalCases", 0] }, { $multiply: [ { $divide: ["$totalDeaths", "$
    totalCases"] }, 100 ] }, 0 ] } } }, { $sort: { totalCases: -1 } }, { $
    out: "totals_per_continent_agg" } ]);

covid_db> db.totals_per_continent_agg.findOne()
{
  _id: 'America',
  totalCases: 30887593,
  totalDeaths: 785420,
  CFR: 2.5428332987941142
}
```

Listing 9: Similar to the previous command, the purpose here is to calculate the total cases, deaths and CFR per continent – instead of country: simply changing the `\protect\T1\textdollargroup` key to `continentExp`. Projecting the new `totals_per_continent_agg` collection with the `findOne()` formula.

```
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % mongoexport --db covid_db --collection totals_per_continent_agg --out totals_per_continent_agg.json --jsonArray
2025-10-27T00:05:56.687+0100  connected to: mongodb://localhost/
2025-10-27T00:05:56.689+0100  exported 6 records
```

Listing 10: Exporting the new `totals_per_continent_agg` collection to file, to visualise and analyse the data at a later stage.

3.3 Global Daily Time Series

```
covid_db> db.covid_data.aggregate([
    {
        $project: {
            cases: 1,
            deaths: 1,
            date: {
                $dateFromString: {
                    dateString: "$dateRep",
                    format: "%d/%m/%Y"
                }
            }
        }
    },
    {
        $group: {
            _id: "$date",
            globalCases: { $sum: "$cases" },
            globalDeaths: { $sum: "$deaths" }
        }
    },
    {
        $sort: { _id: 1 }
    },
    {
        $out: "global_daily_ts_agg"
    }
]);

covid_db> db.global_daily_ts_agg.findOne()
{
  _id: ISODate('2019-12-31T00:00:00.000Z'),
  globalCases: 27,
  globalDeaths: 0
}
```

Listing 11: This pipeline first uses `\protect\T1\textdollardateFromString` to convert the `dateRep` string into BSON dates, so that MongoDB can correctly understand/sort based on chronological order. Much like before, the result is stored in a new collection named `global_daily_ts_agg`, of which a single record is projected using the `findOne()` formula.

```
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % mongoexport --db covid_db --collection global_daily_ts_agg --out global_daily_ts_agg.json --jsonArray
```

```
2025-10-27T00:29:03.765+0100  connected to: mongodb://localhost/
2025-10-27T00:29:03.774+0100  exported 350 records
```

Listing 12: Exporting the new `global_daily_ts_agg` collection to file, to visualise and analyse the data at a later stage.

4 Data Analysis with Hadoop-Python streaming for MR

To compare execution time, the same four metrics were computed using the Hadoop / Python Streaming framework used in the previous coursework assignment. Importantly, since MongoDB commands were executed instanteneously, this comparison was an attempt to see if on the modest 62k record dataset, Hadoop – due to its complexity – can be slower than MongoDB; which we expect to tbe the case.

```
# Load JSON file into HDFS
hdfs dfs -mkdir -p /user/covid/input
hdfs dfs -put /Users/chrismitsacopoulos/Desktop/covid19_nd.json /user/covid/input/

# Make global reducer executable
chmod +x reducer.py
```

Listing 13: `chmod` update on the executable state of the global reducer, as well as setting up the source COVID-19 data within HDFS.

```
#!/usr/bin/env python3
import sys

current_key = None
total_cases = 0
total_deaths = 0

for line in sys.stdin:
    try:
        key, values = line.strip().split('\t', 1)
        cases, deaths = values.split(',', 1)
        cases = int(cases)
        deaths = int(deaths)
    except ValueError:
        continue

    if current_key == key:
        total_cases += cases
        total_deaths += deaths
    else:
        if current_key:
            if total_cases > 0:
                cfr = (total_deaths / total_cases) * 100
            else:
                cfr = 0
            print(f"{current_key}\t{total_cases}\t{total_deaths}\t{cfr:.4f}")

```

```
        current_key = key
        total_cases = cases
        total_deaths = deaths

if current_key == key:
    if total_cases > 0:
        cfr = (total_deaths / total_cases) * 100
    else:
        cfr = 0
    print(f"{current_key}\t{total_cases}\t{total_deaths}\t{cfr:.4f}")
```

Listing 14: Based on reducer_ex5.6.py from the second report, which sums totals. Simple key changes enable it to handle cases and deaths at the same time, while also calculating the Case Fatality Rate (CFR) at the end, just like the MongoDB finalize function. Additional data checks were added, to break operations upon VauleErrors.

4.1 Totals Cases and Deaths per Country with CFR

```
#!/usr/bin/env python3
import sys
import json

for line in sys.stdin:
    try:
        doc = json.loads(line)
        country = doc.get('countriesAndTerritories')
        cases = doc.get('cases', 0)
        deaths = doc.get('deaths', 0)

        if country:
            print(f"{country}\t{cases},{deaths}")
    except json.JSONDecodeError:
        continue
```

Listing 15: Identical logic to the `mapper.py` scripts used in the second report. To identify total deaths, cases and CFR per country, the most important modification was setting `countriesAndTerritories` as the main key.

```
# Make the mapper executable
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % chmod +x mapper_
    country.py
# Run the combinatorial Hadoop/Python streaming command, capture execution
    time and print log to file
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % time hadoop jar $(brew
     --prefix hadoop)/libexec/share/hadoop/tools/lib/hadoop-streaming-*.jar
    \
-file mapper_country.py \
-file reducer.py \
-mapper 'python3 mapper_country.py' \
-reducer 'python3 reducer.py' \
-input /user/covid/input/covid19_nd.json \
-output /user/covid/output_country > job_log_country.txt 2>&1
hadoop jar  -file mapper_country.py -file reducer.py -mapper  -reducer  -
    inpu  3.79s user 0.34s system 174% cpu 2.365 total
# Fetch MR result from HDFS and print to working directory
```

```
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -cat /user/
    covid/output_country/part-00000 > MR_streaming_country_output.txt
```

Listing 16: The `time` shell utility was added to the beginning of the command to return a summary of the system resources used (i.e. systme, user, and sys time) with standard error, following the completion of the Hadoop command. As seen before, the ">" operator redirects the standard output (stdout) to file, using the `2>&1` grammar with the ">" operator, the shell redicrects the standard error (stderr, which is file descriptor 2) to the same place as standard output (&1); thereby printing a text file of choice which can contain the actual Hadoop operation logs and not the shell's record of a local host connection to Hadoop.

4.2 Totals Cases and Deaths per Continent with CFR

```
...
        if continent:
            print(f"{continent}\t{cases},{deaths}") # Change stdout of the
    mapper
...
```

Listing 17: Observe the only important changes from the previous mapper used to obtain counts per country; change of key to identify continent rather than country, reducer handles the computional aspect of this exercise.

```
# Make modified mapper executable
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % chmod +x mapper_
    continent.py
# Remove previous output directory since its no longer needed
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -rm -r /user/
    covid/output_country
2025-10-27 13:44:05,369 WARN util.NativeCodeLoader: Unable to load native-
    hadoop library for your platform... using builtin-java classes where
    applicable
# Execute MR Hadoop/Python streaming job capturing time and printing log to
     file
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % time hadoop jar $(brew
     --prefix hadoop)/libexec/share/hadoop/tools/lib/hadoop-streaming-*.jar
    \
-file mapper_continent.py \
-file reducer.py \
-mapper 'python3 mapper_continent.py' \
-reducer 'python3 reducer.py' \
-input /user/covid/input/covid19_nd.json \
-output /user/covid/output_continent > job_log_continent.txt 2>&1
hadoop jar  -file mapper_continent.py -file reducer.py -mapper  -reducer
        4.00s user 0.37s system 118% cpu 3.685 total
# Fetch MR result from HDFS and print to working directory
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -cat /user/
    covid/output_continent/part-00000 > MR_streaming_continent_output.txt
```

Listing 18: Executing the MR job with same "recipe" used prior, since this method worked for the second report of this module, much of the logic has been repeated.

4.3 Global Daily Time Series

```python
#!/usr/bin/env python3
import sys
import json

for line in sys.stdin:
    try:
        doc = json.loads(line)
        date = doc.get('dateRep')
        cases = doc.get('cases', 0)
        deaths = doc.get('deaths', 0)

        if date:
            print(f"{date}\t{cases},{deaths}")
    except json.JSONDecodeError:
        continue
```

Listing 19: Change of key to dateRep

```
# Make modified mapper executable
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % chmod +x mapper_date.
    py
# Remove previous output directory since its no longer needed
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -rm -r /user/
    covid/output_continent
2025-10-27 13:45:31,984 WARN util.NativeCodeLoader: Unable to load native-
    hadoop library for your platform... using builtin-java classes where
    applicable
Deleted /user/covid/output_continent
# Execute MR Hadoop/Python streaming job capturing time and printing log to
     file
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % time hadoop jar $(brew
     --prefix hadoop)/libexec/share/hadoop/tools/lib/hadoop-streaming-*.jar
    \
-file mapper_date.py \
-file reducer.py \
-mapper 'python3 mapper_date.py' \
-reducer 'python3 reducer.py' \
-input /user/covid/input/covid19_nd.json \
-output /user/covid/output_date > job_log_dates.txt 2>&1
hadoop jar  -file mapper_date.py -file reducer.py -mapper  -reducer  -input
        3.81s user 0.35s system 177% cpu 2.344 total
# Fetch MR result from HDFS and print to working directory
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -cat /user/
    covid/output_date/part-00000 > MR_streaming_date_output.txt
```

Listing 20: Executing the MR job with same "recipe" used prior, since this method worked for the second report of this module, much of the logic has been repeated.

5 Experimenting with Spark

5.1 Setting up Pyspark to reproduce the MongoDB and Hadoop tasks

Note that Java v.17 had to be installed alongside the already existing Java v.11, in order for Spark to work correctly. This led to serious issues with version mismatches, as Hadoop (and the HDFS backend) cannot work without Java v.11 - but Spark cannot work without Java v.17 and the two need to share binaries. As such, small modifications had to be made to what are already near identical commands, to those issued for Hadoop.

The fact that the two programmes were able to co-exist in this convoluted Java environment should be considered a **miracle**.

```
# Create environment to install Python API + extras
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % python3 -m venv
    project
# Load environment
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % source project/bin/
    activate
# Install Python API and JSON handlers / data visualisers
chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % pip install pandas
    seaborn plotly pyspark
# Install Spark globally with Homebrew
(project) chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % brew install
     apache-spark
```

Listing 21: Creating an environment to donwload the Python API for Spark, as well as data visualisation libraries for donwstream analysis. Lastly, installing Spark with Homebrew.

In order to use Spark similar to how it is used in the course material, as well as to leverage the easy of use of Python, a script `spark_analysis.py` was drafted to implement the Spark Python API and help automate the MR process. Specifically, this script attempts to implement the RDD workflow presented in the slides `CoursCloudComputingForM2Genomics-part5`:

- **Making RDD:** Creating a `SparkContext` (sc) first, then uses `sc.textFile()` to create the base RDD from HDFS.
- **Apply Transformations:** Applies a chain of "lazy" transformations, including `.map()` (using `lambda` functions) to parse JSON and structure key-value pairs, and `.reduceByKey()` to aggregate the data, similar to the WordCount example (Slide 11).
- **Apply Action:** Finally, it calls `.saveAsTextFile()`, which is an **Action** that triggers the computation of all preceding transformations.

```
#!/usr/bin/env python3
import sys
import json
from pyspark import SparkContext

def parse_line(line):
```

```
    # parse a line of JSON, skipping bad lines
    try:
        return json.loads(line)
    except:
        return {} # Return empty dict to avoid errors

def sum_stats(a, b):
    # "reducer" function sums (cases, deaths) tuples
    return (a[0] + b[0], a[1] + b[1])

def calculate_cfr(record):
    # takes a final record (key, (total_cases, total_deaths))
    #  returns a formatted string with the CFR.
    key = record[0]
    cases = record[1][0]
    deaths = record[1][1]

    if cases > 0:
        cfr = (deaths / cases) * 100
    else:
        cfr = 0

    # Return a tab-separated string, ready for the output file
    return f"{key}\t{cases}\t{deaths}\t{cfr:.4f}"

def main(sc, input_path, key_field, output_path):
    # Create an RDD from HDFS file.
    #  "lazy" operation -- placeholder esque
    lines = sc.textFile(input_path)
    # Parse each line (string) into a dictionary
    data = lines.map(parse_line)
    # Map to (key, (cases, deaths)) pairs --> dynamic keep in mind
    keyed_data = data.map(lambda doc: (doc.get(key_field, "null_key"),
                                       (doc.get("cases", 0), doc.get("
    deaths", 0))))
    # sum / reduce all stats for each key
    totals = keyed_data.reduceByKey(sum_stats)
    final_results = totals.map(calculate_cfr)

    # Execute all (lazy) transforms and save to HDFS.
    final_results.saveAsTextFile(output_path)


if __name__ == "__main__":
    if len(sys.argv) != 4:
        # first argument is input file, second ex. "countriesAndTerritories
    " third is output dir
        print("Usage: spark_analysis.py <input_path> <key_field> <output_
    path>", file=sys.stderr)
        sys.exit(-1)

    # Create a SparkContext. 'local[*]' tells Spark to use all CPU cores
    available
    # This is the main connection to the Spark cluster
    sc = SparkContext(appName="SparkCovidAnalysis", master="local[*]")

    input_path = sys.argv[1]
    key_field = sys.argv[2]
```

```
    output_path = sys.argv[3]

    main(sc, input_path, key_field, output_path)

    # Always stop the context when done
    sc.stop()
```

Listing 22: The `spark_analysis.py` script, which accepts command-line arguments to change the key / centre of analysis (ex. `countriesAndTerritories`). The script was made executable using `chmod+xspark_analysis.py`.

5.2 Total Cases and Deaths per Country with CFR

```
#Running Spark with spark_analysis.py
(project) chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % time spark-submit spark_analysis.py \
    hdfs://localhost:9000/user/covid/input/covid19_nd.json \
    countriesAndTerritories \
    hdfs://localhost:9000/user/covid/output_spark_country > spark_countries.txt 2>&1
spark-submit spark_analysis.py  countriesAndTerritories  > spark_countries.tx  6.89s user 0.43s system 133% cpu 5.473 total
# Print result to working directory
(project) chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -cat /user/covid/output_spark_country/part-00000 > MR_SPARK_country_output.txt
```

Listing 23: Re-using the same logic as with the Hadoop commands. Note the inclusion of `hdfs://localhost:9000` which gave HDFS access to the working directory as Java version mismatches led to `DataNode` errors.

5.3 Total Cases and Deaths per Continent with CFR

```
#Running Spark with spark_analysis.py
(project) chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % time spark-submit spark_analysis.py \
    hdfs://localhost:9000/user/covid/input/covid19_nd.json \
    continentExp \
    hdfs://localhost:9000/user/covid/output_spark_continent > spark_continents.txt 2>&1
spark-submit spark_analysis.py  continentExp  > spark_continents.txt 2>&1  6.78s user 0.49s system 131% cpu 5.553 total
#Print result to working directory
(project) chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -cat /user/covid/output_spark_continent/part-00000 > MR_SPARK_continents_output.txt
```

Listing 24: Using Spark for the continents based search. Same logic as before.

5.4 Global Daily Time Series

```
#Running Spark with spark_analysis.py
(project) chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % time spark-submit spark_analysis.py \
    hdfs://localhost:9000/user/covid/input/covid19_nd.json \
    dateRep \
    hdfs://localhost:9000/user/covid/output_spark_date > spark_dates.txt 2>&1
spark-submit spark_analysis.py  dateRep  > spark_dates.txt 2>&1  6.76s user 0.46s system 128% cpu 5.607 total
#Print result to working directory
(project) chrismitsacopoulos@Chriss-MacBook-Air BFD_Report_3 % hdfs dfs -cat /user/covid/output_spark_date/part-00000 > MR_SPARK_dates_output.txt
```

Listing 25: Using Spark for the dates based search. Same logic as before.

6 Comparing MR execution times across the different platforms

6.1 MongoDB Performance Summary

The four aggregation pipelines were executed using the JavaScript timing script to capture the precise server-side (i.e. local-host) execution time. The results, as summarised in Table **??**, demonstrate that all queries were extremely fast, **executing within $\approx$ 40 ±10ms for each MongoDB command**. Observe below the JavaScript code used to obtained these metrics on the MongoDB execution time:

- Totals & CFR per Country (ms): 38
- Totals & CFR per Continent (ms): 34
- Global Daily Time Series & CFR (ms): 51


```
        },
        "executionStats": {
          "executionSuccess": true,
          "nReturned": 350,
          "executionTimeMillis": 51,
          "totalKeysExamined": 0,
          "totalDocsExamined": 61900,
          "executionStages": {
            "stage": "project",
            "planNodeId": 3,
            "nReturned": 350,
            "executionTimeMillisEstimate": 38,
            "opens": 1,
            "closes": 1,
            "saveState": 6,
```

```
            "restoreState": 5,
            "isEOF": 1,
            "projections": {
              "15": "newObj(\"_id\", s10, \"globalCases\", s13, \"
    globalDeaths\", s14) "
            },
```

Listing 26: Snippet of the "explain stage" array for `GlobalDailyTimeSeries&CFR(ms)` generated by the automated execution time check. Note the `executionSuccess` and `"executionTimeMillis":51` data fields, these were taken into account when comparing the platforms. Find the entire search result saved as `time_results.txt` in the supplementary materials.

```
const conn = new Mongo("mongodb://localhost:27017");
const db = conn.getDB("covid_db");

const docCount = db.covid_data.countDocuments();
if (docCount === 0) {
    print("CRITICAL ERROR: No documents found in 'covid_data' collection
    within 'covid_db'.");
} else {
    print(`SUCCESS: Found ${docCount} documents. Proceeding with tests...\n
    `);
}

function runAndPrintStats(pipelineName, pipeline) {
    print(`--- ${pipelineName} (ms) ---`);

    const command = {
        explain: { aggregate: "covid_data", pipeline: pipeline, cursor: {}
    },
        verbosity: "executionStats"
    };

    const stats = db.runCommand(command);

    if (stats && stats.executionStats && stats.executionStats.
    executionTimeMillis !== undefined) {
        print(stats.executionStats.executionTimeMillis);
    } else {
        print("ERROR: Could not find 'executionTimeMillis'. Server returned
    :"); //For some reason this executes all the time
        print(JSON.stringify(stats, null, 2));
    }
}

const pipeline1 = [
    { $group: { _id: "$countriesAndTerritories", totalCases: { $sum: "$
    cases" }, totalDeaths: { $sum: "$deaths" } } },
    { $project: { _id: 1, totalCases: 1, totalDeaths: 1, CFR: { $cond: [ {
    $gt: ["$totalCases", 0] }, { $multiply: [ { $divide: ["$totalDeaths", "$
    totalCases"] }, 100 ] }, 0 ] } } },
    { $sort: { totalCases: -1 } }
];

const pipeline2 = [
    { $group: { _id: "$continentExp", totalCases: { $sum: "$cases" },
    totalDeaths: { $sum: "$deaths" } } },
```

```
    { $project: { _id: 1, totalCases: 1, totalDeaths: 1, CFR: { $cond: [ { $gt: ["$totalCases", 0] }, { $multiply: [ { $divide: ["$totalDeaths", "$totalCases"] }, 100 ] }, 0 ] } } },
    { $sort: { totalCases: -1 } }
];

const pipeline3 = [
    { $project: { cases: 1, deaths: 1, date: { $dateFromString: { dateString: "$dateRep", format: "%d/%m/%Y" } } } },
    { $group: { _id: "$date", globalCases: { $sum: "$cases" }, globalDeaths: { $sum: "$deaths" } } },
    { $project: { _id: 1, globalCases: 1, globalDeaths: 1, CFR: { $cond: [ { $gt: ["$globalCases", 0] }, { $multiply: [ { $divide: ["$globalDeaths", "$globalCases"] }, 100 ] }, 0 ] } } },
    { $sort: { _id: 1 } }
];

if (docCount > 0) {
    runAndPrintStats("1. Totals & CFR per Country", pipeline1);
    runAndPrintStats("2. Totals & CFR per Continent", pipeline2);
    runAndPrintStats("3. Global Daily Time Series & CFR", pipeline3);
}
```

Listing 27: The above JavaScript code was executed using `mongosh--filetime_test_mongo.js` with the same working directory, used in the previous sections of this report. The important inclusion in this code are the connection and database variables; `constconn` and `constdb` respectively, which if not explicitly stated in the JS code, the server would return an error – even when explicitly denoting `covid_db` in the `mongosh` shell operator.

6.2 Hadoop Performance Summary

As discussed in the Hadoop-Python streaming job sections, the `time` shell command was added to the beginning of all commands to capture the execution time, as well as resources used by the system to finalise execution of the command (i.e. CPU usage percentage etc.). Specifically, the summary of execution times:

- Job 1 (Per-Country) took **2.365 seconds**.
- Job 2 (Per-Continent) took **3.685 seconds**.
- Job 3 (Daily Time Series) took **2.344 seconds**.

6.3 Spark Performance Summary

Using the `time` shell command in the beginning of all Spark commands – much like with Hadoop – the following results were obtained:

- Job 1 (Per-Country) took **5.473 seconds**.

- Job 2 (Per-Continent) took **5.553 seconds**.
- Job 3 (Daily Time Series) took **5.607 seconds**.

6.4 Discussion

The MongoDB aggregations consistently executed with a local-host server time between 34-51ms. Comparatively, the Hadoop-Python MR streaming jobs required a minimum of 2.3 to 3.7 seconds of real time. This makes the MongoDB Aggregation Pipeline approximately 60 (lower end) to 90 times (upper end) faster than the Hadoop tasks. The performance gap is even more obvious with Spark, which was unexpectedly the slowest. As shown in Table 2, MongoDB was more than 100x faster than Spark at executing each task. These differences of course, have a lot to do with the nature of these data management systems.

Table 2: Performance Comparison: MongoDB vs. Hadoop

Aggregation Task	**MongoDB (ms)**	**Hadoop (s)**	**Spark (s)**
Totals & CFR per Country	38	2.365	5.473
Totals & CFR per Continent	34	3.685	5.553
Global Daily Time Series & CFR	51	2.344	5.607

For one, MongoDB is designed for high-throughput, low-latency database operations. For a database with 61900 records, any modern computer (ex. MacBook Air M1) can handle the size of this dataset on RAM, foregoing any possible bottlenecks of I/O operations on the computer's drive. The highly-optimised native C++ backend of MongoDB also ensures that searches of this mangitude take place in the span of milliseconds and not seconds.

Comparatively, Hadoop is built for batch processing of datasets where the number of records in a dataset would be highly inneficient to compute using conventional database platforms such as PostgresSQL or NoSQL based – MongoDB. One must also consider that Hadoop operates on multiple daemons accesing both disk and RAM memory (I/O bottlenecks), also having to deal with Python scripts during MR streaming (Python being a scripting language with a slow interpreter).

With reference to Table 2, the only Hadoop task which didn't take $\approx$ 2.35s to execute was the continent based search. On a logical basis, this result seems paradoxical as the number of keys to work with are only 6, reducing the overall number of computations needed to complete the task. This logical assumption is confirmed by MongoDB's performance on the same task, which it executed the fastest out of the three tasks, within 34ms. Another paradoxical observation, is that the fastest task for Hadoop is the slowest one for MongoDB; referring to the Global Daily Time Series in Table 2. The only explanation to this can be differences in algorithmic search logic concerning DD/MM/YYYY date formats.

Of course these paradoxical observations could be owed to system errors / delays which might have randomly occured when running the experiments. Rerunning MongoDB was easy and

the only differences observed were in the range of ± ≈ 5ms depending on the device was connected to the power supply (increasing CPU voltage from the battery). Repeating the searches in Hadoop did not yield significantly different results – only small improvements with power supply draw – which cannot be explained; especially why the Totals & CFR per Continent takes longer than the others.

Even more confusing was the performance of Spark, which executed at the slowest speeds of ≈ 5.5s. The lecture slides state that Spark is designed to be "up to a hundred times faster than Hadoop MapReduce applications" because it "uses in-memory primitives" and "uses the central memory of the cluster machines much better", avoiding the disk I/O which bottlenecks Hadoop. From sources on the internet, the only explanation as to why Spark could have been so slow compared to Hadoop – in the report's usage example – can be summarised to startup overhead. Specifically, for every Spark job, the `spark-submit` script must launch a full driver program, create a `SparkContext`, connect to HDFS, and coordinate its local executors; by the time this startup has been executed, Hadoop has already finished streaming to the Python based mappers and reducers, printing an output to the HDFS.

6.5 Conclusion

MongoDB is by far the best choice for efficiently analysing a dataset of this size, requiring the least amount of execution time, preparation, coding and debugging. Hadoop and Spark are designed for larger datasets, the effort required to run them far exceed the benefits in this usage case.

7 Visualising the MR results on the ECDC dataset

The graphs that follow depict the data returned by all MR tests performed across either MongoDB, Hadoop-Python or Spark-Python. In order to reproduce these interactive figures on your device, ensure you have:

- Installed the `pandas`, `plotly`, and `seaborn` Python libraries.
- Have `visualise.py`, `global_daily_ts_agg.json`, `totals_per_country_with_code.json`, and `totals_per_continent_agg.json` within the same directory. These are all supplied in the `.zip` file submitted alongside this report.

Reading this you will probably notice the inclusion of a new file named `totals_per_country_with_code.json`. This was generated by re-running the original MongoDB command for the "Totals & CFR per country" but switching from storing country names to storing their ISO Codes. This enabled the Plotly extensions (choropleth) to create the interactive world map (See Figure 1), on which hovering over a country can give you its MR-derived metadata (i.e. Total Cases, Total Deaths and CFR).

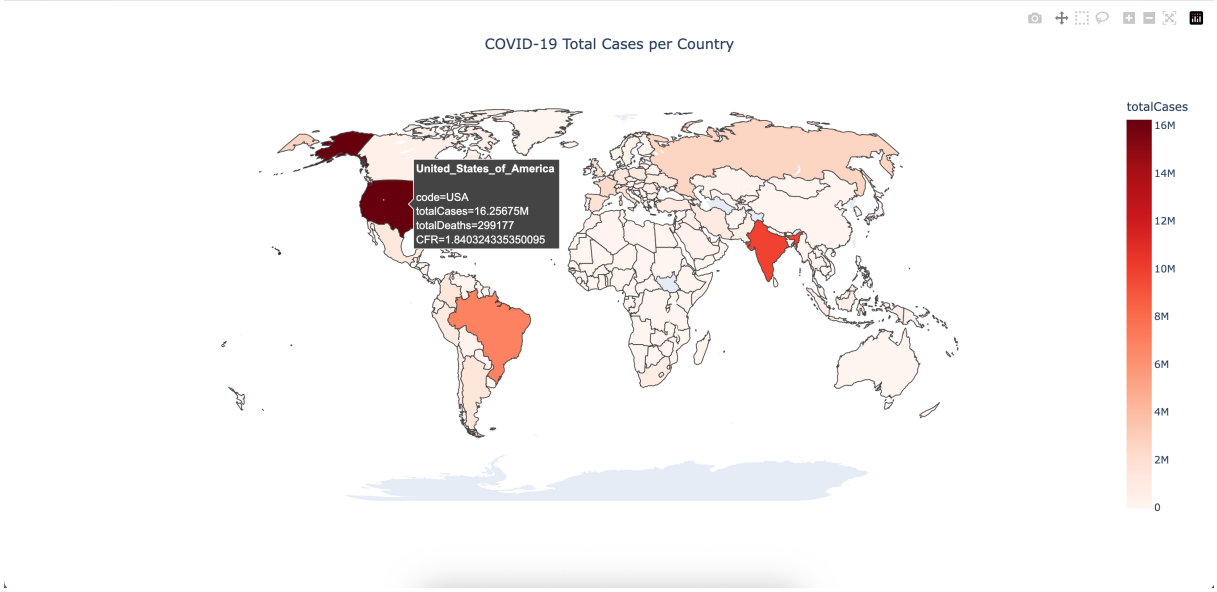

Figure 1: An interactive choropleth map generated with Plotly, visualising the total number of COVID-19 cases per country. The colour intensity (from white to dark red) correlates with the total cases computed for that country. The hover-over metadata provides specific metrics per country, including the ISO code, total cases, deaths, and the calculated CFR.

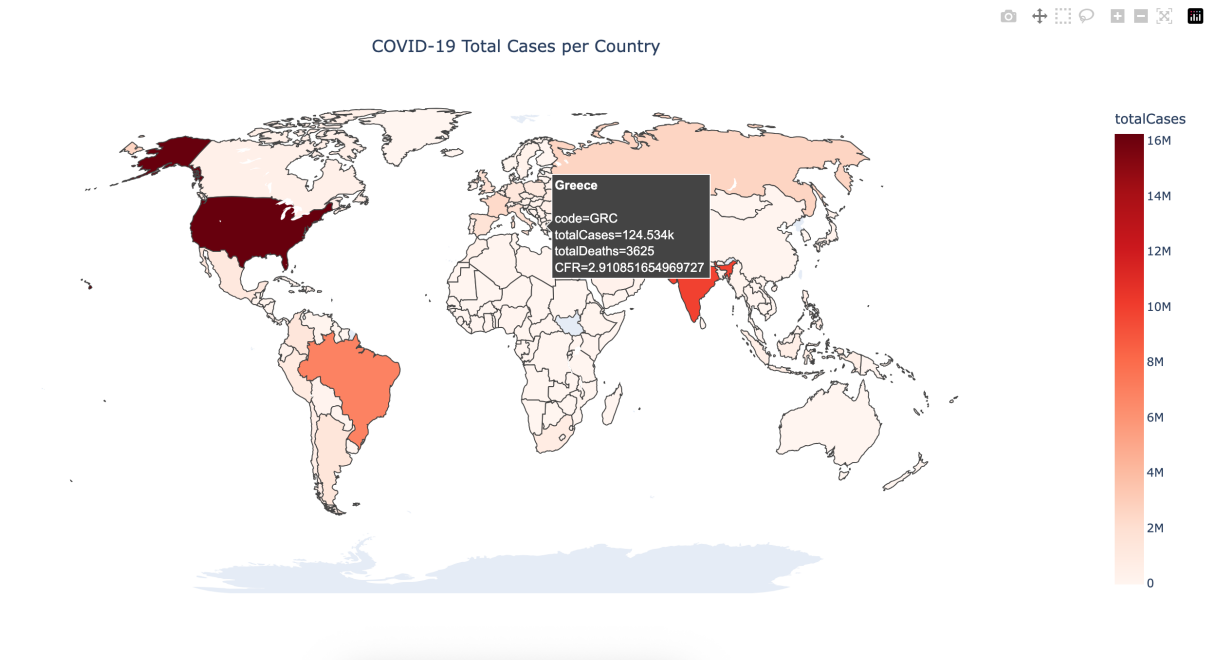

Figure 2: Another screenshot of the same map in Figure 1. The cursor in this image is hovering above Greece, to display its MR computed metadata.

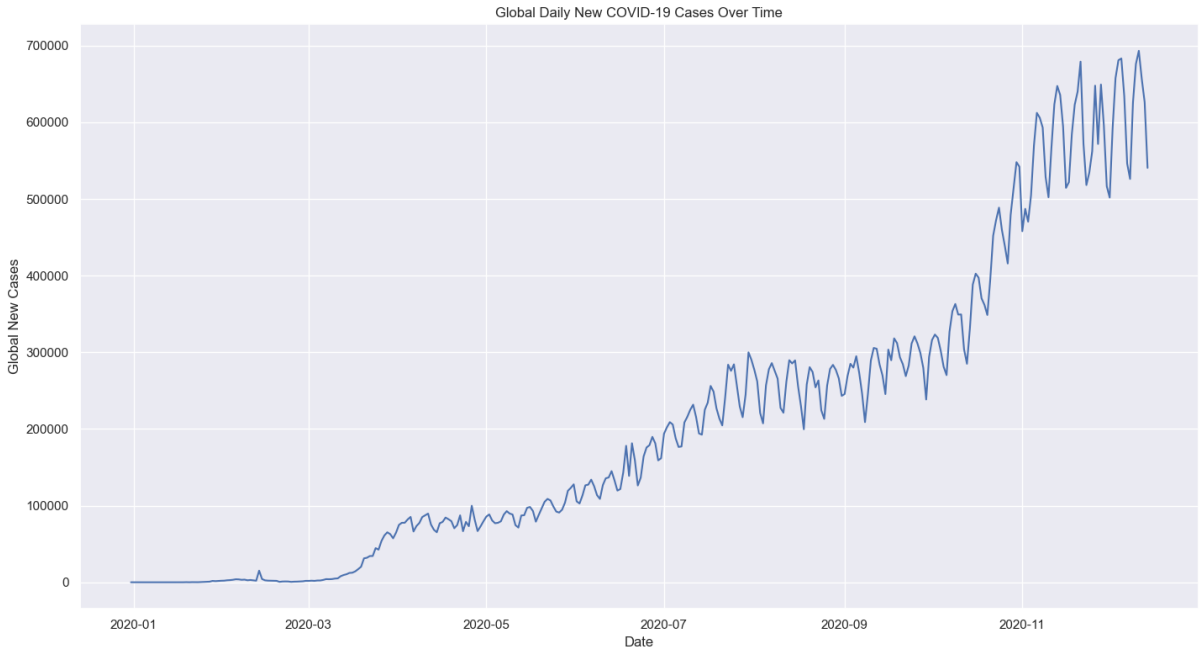

Figure 3: A time-series line chart, plotted with Seaborn, illustrating the global daily new reported cases of COVID-19. The plot clearly visualises the "waves" of the pandemic, showing a significant increase in cases during the latter half of 2020.

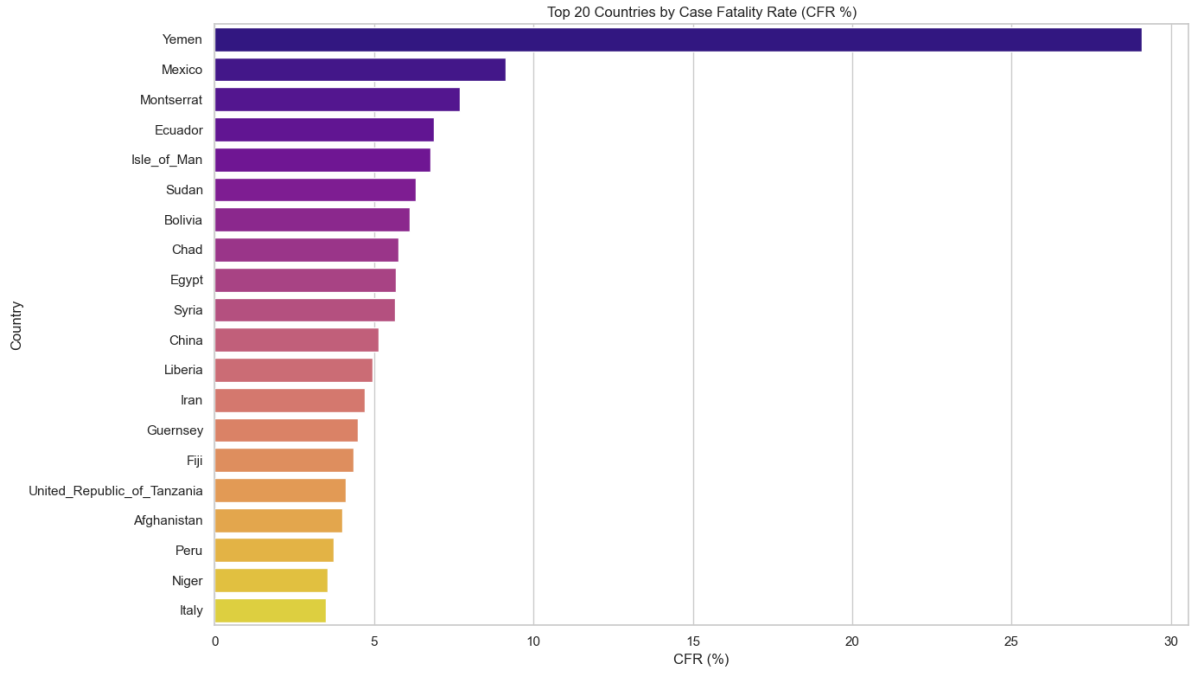

Figure 4: A horizontal bar chart generated with Seaborn, displaying the 20 countries with the highest CFR; highlighting the variance in mortality percentage across countries. Student's note: Would be interesting to examine GDP and / or average age of death, to observe their correlation on CFR.

7.1 Python Code to reproduce figures

Find a description of the Python code at the bottom of the code listing:

```
import pandas as pd
import plotly.express as px
import seaborn as sns
import matplotlib.pyplot as plt

def load_data():
    try:
        df_country = pd.read_json('totals_per_country_with_code.json')
        df_dates = pd.read_json('global_daily_ts_agg.json')
    except ValueError as e:
        print(f"Error loading JSON: {e}")
        print("Please ensure your JSON files are in the same directory.")
        return None, None

    date_strings = df_dates['_id'].apply(lambda x: x['$date'])
    df_dates['_id'] = pd.to_datetime(date_strings)
    df_dates.rename(columns={'_id': 'date'}, inplace=True)
    df_dates.sort_values(by='date', inplace=True)

    return df_country, df_dates

def plot_interactive_map(df_country):
    fig = px.choropleth(
        df_country,
        locations="code",
        color="totalCases",
        hover_name="country",
        hover_data=["totalCases", "totalDeaths", "CFR"],
        projection="natural earth",
        title="COVID-19 Total Cases per Country",
        color_continuous_scale=px.colors.sequential.Reds
    )

    fig.update_layout(
        title_x=0.5,
        geo=dict(
            showframe=False,
            showcoastlines=False
        )
    )

    print("Displaying interactive map. A browser window will open.")
    fig.show()

def plot_static_charts(df_country):
    df_top_20_cfr = df_country.nlargest(20, 'CFR')

    plt.figure(figsize=(14, 8))
    ax2 = sns.barplot(
        data=df_top_20_cfr,
        x="CFR",
        y="country",
        palette="plasma"
    )
```

```
    ax2.set_title('Top 20 Countries by Case Fatality Rate (CFR %)')
    ax2.set_xlabel('CFR (%)')
    ax2.set_ylabel('Country')

    print("Displaying Top 20 CFR bar chart.")
    plt.tight_layout()
    plt.show()

def plot_time_series(df_dates):
    sns.set_theme(style="darkgrid")

    plt.figure(figsize=(16, 8))
    ax = sns.lineplot(
        data=df_dates,
        x='date',
        y='globalCases'
    )
    ax.set_title('Global Daily New COVID-19 Cases Over Time')
    ax.set_xlabel('Date')
    ax.set_ylabel('Global New Cases')

    print("Displaying global daily cases time series.")
    plt.tight_layout()
    plt.show()

def main():
    df_country, df_dates = load_data()

    if df_country is not None and df_dates is not None:
        plot_interactive_map(df_country)
        plot_static_charts(df_country)
        plot_time_series(df_dates)

if __name__ == "__main__":
    main()
```

Listing 28: The entire code from `visualise.py`. Paste this code into a new Python file with the necessary JSON files in the same directory (`global_daily_ts_agg.json`, `totals_per_country_with_code.json`, and `totals_per_continent_agg.json`), to get all of the graphs depicted in this report – in plotly **INTERACTIVE FORMAT** (HTML for the map figure and PNG for the rest).